

Syntaxer AI

AI-Powered Code Debugging & Security Analysis Platform

Author Jesvin Version 1.0 · June 2026	Live syntaxer-ai.onrender.com Repo github.com/Jesvin-py/Syntaxer-AI
--	--

1. Introduction

Syntaxer AI is an AI-powered web application that helps students and early-career developers analyze code for bugs, security vulnerabilities, and educational insights. Unlike chatbot-style tools that return unstructured prose, Syntaxer AI enforces a strict JSON output schema from the LLM, enabling the frontend to render structured, actionable reports.

The system operates across three analysis modes:

- Debug — Root cause identification, step-by-step fix guidance, and corrected code generation
- Security — Vulnerability scanning with CRITICAL / HIGH / MEDIUM / LOW severity classification
- Shield Scan — Combined analysis with a learning panel explaining underlying concepts

2. Problem Statement

Two critical pain points affect students and junior developers:

- Debugging inefficiency — Error messages are cryptic, Stack Overflow answers are inconsistent, and beginners spend hours on problems with known patterns
- Security blindness — Code copied from AI tools or GitHub often contains SQL injection risks, hardcoded credentials, and missing input validation that go unnoticed until production

Existing tools address one or the other. Syntaxer AI combines both in a single structured interface purpose-built for the learning stage of a developer's career.

3. Objectives

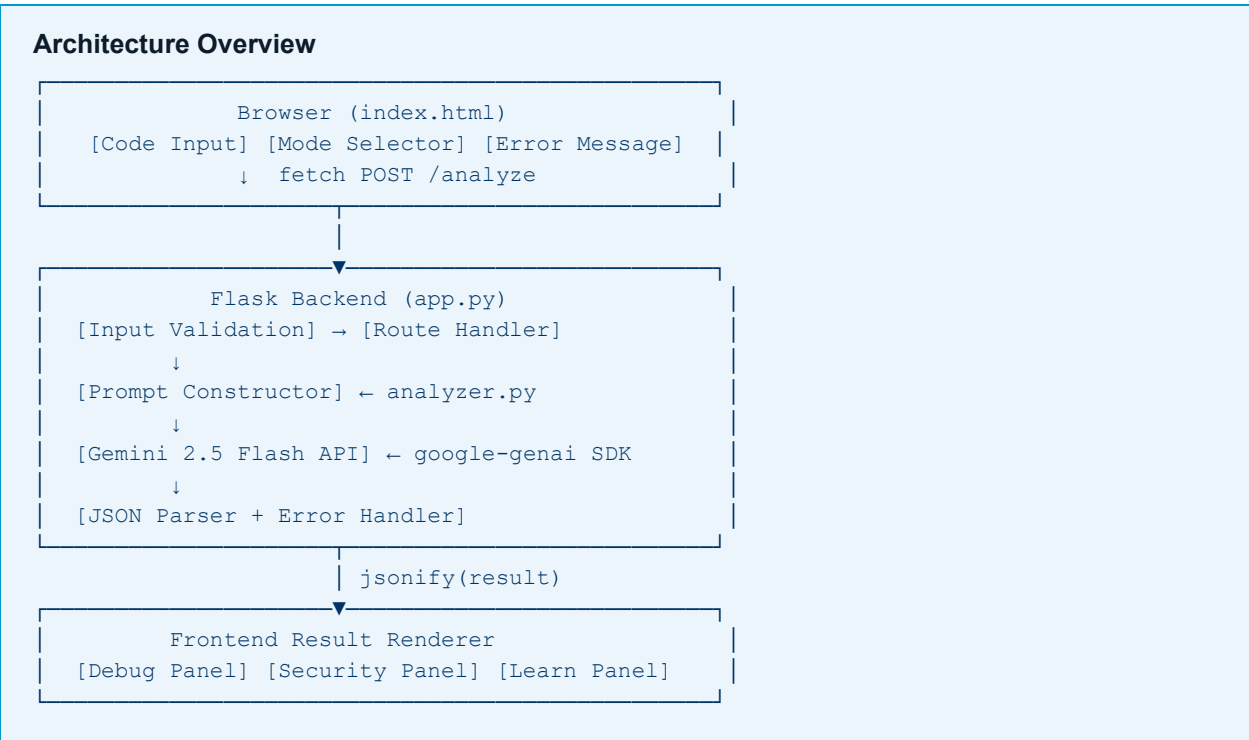
1. Deliver debug analysis, security scanning, and educational insight in a single API call
2. Return structured, machine-renderable JSON output — not chatbot prose
3. Keep the architecture simple enough to deploy and extend without a database or authentication layer
4. Handle external API unreliability gracefully with exponential backoff retry logic
5. Provide a UI experience that matches production SaaS standards, not a student prototype

4. Technology Stack Justification

Technology	Justification	Trade-off
Flask	Lightweight, zero boilerplate. A single POST route is all this system requires. FastAPI adds async complexity with no benefit at this scope.	Single-threaded by default; concurrent users would need Gunicorn workers
Gemini 2.5 Flash	Best balance of speed, instruction-following, and structured JSON output. Flash-tier latency (3–6s) is acceptable for synchronous analysis.	Occasional 503 demand spikes require retry handling
google-genai SDK	The current supported SDK. Replaced deprecated google-generativeai mid-project after a FutureWarning was raised.	Newer API; less community documentation available
Vanilla JavaScript	No build toolchain, no bundler, no framework overhead. One page, one API call, three rendering functions.	No component reuse; scaling UI features requires careful DOM management
Render.com	Free tier with automatic GitHub deploy on every push. Zero infrastructure management.	Cold start delay (~30–60s) after 15 minutes of inactivity
No Database	Analysis results are stateless. Users paste code, read results, and move on. A DB adds infrastructure complexity with no MVP user-facing benefit.	No history, no caching, no user accounts

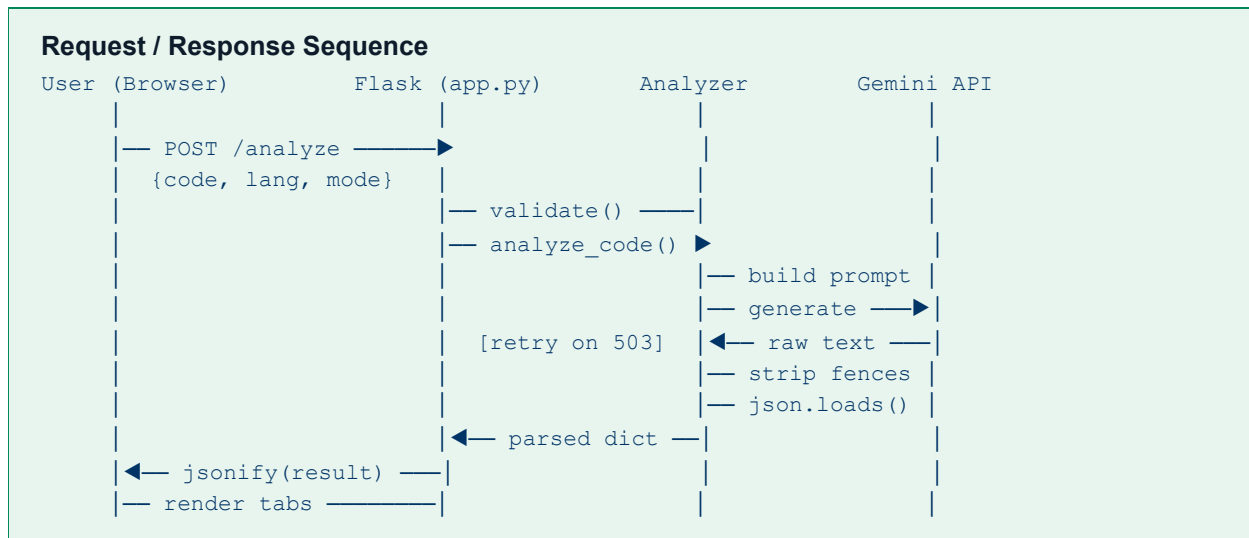
5. High-Level Architecture

The architecture follows an MVC-lite pattern with three distinct layers:



The architecture is deliberately flat. There is no middleware layer, no ORM, no message queue. Every request completes in a single synchronous cycle: receive → validate → prompt → call → parse → respond.

6. Data Flow and Processing Pipeline



Input validation is performed at the Flask layer before any API call. This prevents unnecessary quota consumption on empty or oversized inputs. JSON parsing failures return a structured fallback dict — the frontend always receives a valid response shape.

7. Prompt Engineering Strategy

The most critical design decision in the system is the strict JSON schema enforced through the system prompt in prompts.py.

Why This Matters

LLMs default to prose. A naive prompt returns a paragraph of text with no predictable structure. The frontend cannot render prose as severity badges, collapsible cards, or syntax-highlighted code blocks. The schema contract is what transforms a general-purpose LLM into a structured data pipeline.

Schema Contract

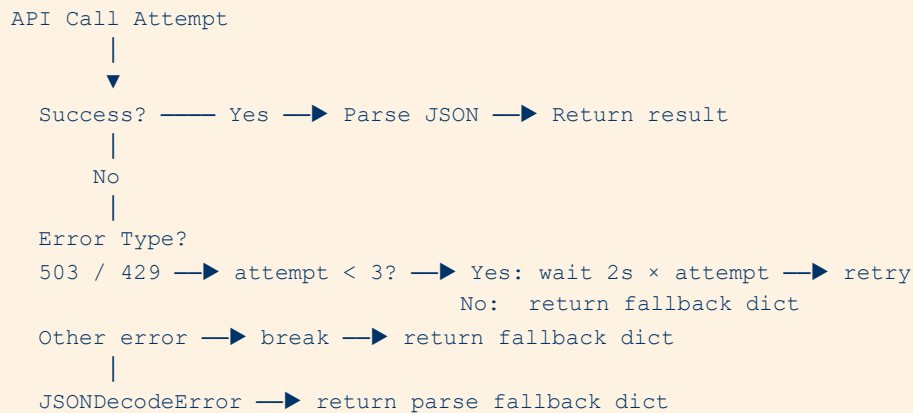
```
{
  "debug": { has_bug, root_cause, explanation, fix_steps[], corrected_code }
  "security": { overall_risk, vulnerabilities[{title, severity, line_hint, explanation, fix}] }
  "learn": { concept, why_it_matters, tip }
}
```

Key Decisions

- temperature: 0.2 — Minimises hallucination; keeps outputs deterministic across similar inputs
- Defensive fence stripping — Gemini occasionally wraps output in ```json blocks despite instructions; analyzer.py strips these before json.loads()
- Mode-aware depth — The mode parameter (debug / security / shield) is included in the user message so Gemini allocates analytical depth appropriately per section

8. Reliability and Error Handling

Retry Flow



Three failure modes are handled explicitly:

- 503 UNAVAILABLE — Gemini server overload. Retried up to 3 times with exponential backoff (2s, 4s, 6s).
- 429 RESOURCE_EXHAUSTED — API rate limit. Same retry path as 503.
- JSONDecodeError — Unexpected LLM output format. Returns a structured fallback so the frontend always receives a valid response shape.

9. Design Decisions and Trade-offs

Decision	Rationale	Trade-off
Stateless, no database	Zero infrastructure complexity for MVP scope	No result history, no caching
Single POST endpoint	Matches the tool's simple input/output contract; mode is a parameter	No granular endpoints per analysis type
Synchronous API calls	Simpler code, easier to debug and trace	Long responses block the thread; acceptable at prototype scale
No authentication	Reduces scope; not needed for a public analysis tool	Open to API quota abuse without rate limiting
Render free tier	Zero cost, auto-deploy on git push	30–60s cold start after inactivity

10. Scalability and Future Improvements

The current architecture handles one user at a time cleanly. Under higher load, the following additions are prioritised:

- Async task queue — Replace synchronous Gemini call with Celery + Redis for non-blocking analysis
- Response caching — Hash code inputs; cache results in Redis to avoid redundant API calls
- Rate limiting — Flask-Limiter middleware per IP to prevent quota abuse
- User sessions and history — SQLite or PostgreSQL to persist past analyses per user
- GitHub integration — Accept repository URLs and analyse files via the GitHub API
- Streaming output — Use Gemini streaming API to progressively render results rather than blocking until completion

11. Security Considerations

- API key isolation — Gemini key stored in .env locally and Render environment dashboard in production. Never committed to the repository (.gitignore enforced).
- Input length validation — Requests exceeding 5,000 characters are rejected at the Flask layer before reaching the API.
- No code execution — Syntaxer AI analyses code as text only. User-submitted code is never executed, eliminating the primary risk surface of a code analysis tool.
- No persistent storage — No user data is retained between sessions.

Known limitation: There is no authentication, CORS restriction, or rate limiting on the /analyze endpoint. Rate limiting and CORS headers are the first production hardening steps before any public launch.

12. Conclusion

Syntaxer AI demonstrates that an LLM can function as a structured data pipeline rather than a chatbot when prompt design enforces a strict output contract. The architecture is intentionally minimal — one Flask route, one external API call, one HTML page — which made it possible to build, test, and deploy a working product within a four-day timeline.

The core engineering insight is that the quality of the system prompt determines the quality of the product. The JSON schema, temperature setting, and defensive parsing together transform a general-purpose LLM into a predictable analysis engine that a frontend can render as a professional tool.

Live: syntaxer-ai.onrender.com | Repository: github.com/Jesvin-py/Syntaxer-AI